

Javascript

A yellow square containing the letters 'JS' in a large, bold, black sans-serif font.

1.Introduction to JavaScript

- What is Javascript ?

JavaScript is a high-level, interpreted programming language primarily used for building interactive and dynamic web pages. It allows developers to add functionality to web pages, create interactive elements, and handle client-side tasks.

JavaScript is an essential technology for web development, as it enables the creation of dynamic and responsive user interfaces.

- Key features of JavaScript include:

Client-Side Scripting:

JavaScript is mainly used for client-side scripting, meaning it runs in the user's web browser rather than on the web server. This allows developers to create dynamic and interactive user interfaces.

Object-Oriented:

JavaScript is an object-oriented language, supporting features like encapsulation, inheritance, and polymorphism. Objects in JavaScript can be defined and manipulated, making it a versatile language for various programming paradigms.

Event-Driven:

JavaScript is often used to respond to user actions or events on a webpage, such as clicks, form submissions, or keyboard inputs. This event-driven nature makes it suitable for creating responsive and interactive user interfaces.

Cross-platform Compatibility:

JavaScript is supported by all major web browsers, making it a cross-platform language. This allows developers to write code that works consistently across different browsers and operating systems.

Asynchronous Programming:

JavaScript supports asynchronous programming through features like callbacks, promises, and async/await. This is crucial for handling tasks that may take time to complete, such as fetching data from a server, without blocking the rest of the program's execution.

Libraries and Frameworks:

There are numerous libraries and frameworks built on top of JavaScript that simplify common tasks and provide additional functionalities. Some popular ones include jQuery, React, Angular, and Vue.js.

- The role of javascript in web development:

Client-Side Scripting:

JavaScript is primarily a client-side scripting language, meaning it runs on the user's browser. It allows developers to

create dynamic and interactive user interfaces by manipulating the Document Object Model (DOM) of a webpage.

Dynamic Content:

JavaScript enables the modification of content on a webpage dynamically. This can include updating text, images, and other elements without requiring a full page reload. This dynamic content update enhances the user experience and responsiveness of web applications.

Event Handling:

JavaScript is used to respond to user interactions, such as clicks, form submissions, and keyboard inputs. Through event handling, developers can create responsive and interactive features, making web applications more user-friendly.

Form Validation:

JavaScript is commonly used for client-side form validation, ensuring that user input meets specific criteria before being submitted to a server. This helps improve data integrity and provides a better user experience by giving instant feedback to users.

AJAX (Asynchronous JavaScript and XML):

JavaScript facilitates asynchronous communication with servers, allowing web pages to fetch or send data in the background without requiring a page reload. This enhances the speed and efficiency of web applications by updating content without disrupting the user's experience.

Browser Compatibility:

JavaScript is supported by all major web browsers, ensuring cross-browser compatibility. This allows developers to write code that works consistently across different browsers, providing a seamless experience for users.

Frameworks and Libraries:

JavaScript has a rich ecosystem of libraries and frameworks, such as jQuery, React, Angular, and Vue.js. These tools simplify common tasks, provide abstractions for complex functionality, and expedite the development process.

Animations and Effects:

JavaScript enables the creation of animations and visual effects on web pages. This enhances the overall user experience by making interfaces more engaging and visually appealing.

Single Page Applications (SPAs):

JavaScript is a key technology in building SPAs, where a single HTML page is loaded initially, and subsequent content changes are managed dynamically without full page reloads. This architecture provides a more fluid and app-like experience.

Server-Side Development (Node.js):

With the advent of Node.js, JavaScript can also be used for server-side development. This allows developers to use the same language on both the client and server sides, streamlining development workflows.

2.Basic of JavaScript

- **Embedding JavaScript in Html**

To embed JavaScript in HTML, you can use the `<script>` element. The `<script>` element is used to include or reference external JavaScript files, as well as to embed JavaScript code directly within an HTML document.

Inline Script:

You can include JavaScript directly within your HTML document using the `<script>` tag in the `<head>` or `<body>` section. Here's an example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Inline Script Example</title>
  <script>
    // Your JavaScript code goes here
    function greet() {
      alert("Hello, World!");
    }
  </script>
</head>
<body>
  <h1>Embedding JavaScript in HTML</h1>
  <button onclick="greet()">Click me</button>
</body>
</html>
```

External Script File:

You can also include JavaScript code from an external file. Create a separate .js file and reference it using the src attribute of the <script> tag:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>External Script Example</title>
  <script src="script.js"></script>
</head>
<body>
  <h1>Embedding JavaScript in HTML</h1>
  <button onclick="greet()">Click me</button>
</body>
</html>
```

- Basic syntax and Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>JavaScript Example</title>
</head>
<body>

  <!-- Your HTML content goes here -->

  <script>
    // Your JavaScript code goes here
    // This is a comment in JavaScript

    // Declaring variables
    let greeting = "Hello, ";
    let name = "World";

    // Concatenating strings
    let message = greeting + name;

    // Displaying a message using alert
    alert(message);

    // Writing to the browser console
    console.log("This message is logged to the console.");

    // Conditional statement
    if (name === "World") {
      document.write("<p>It's the World!</p>");
    } else {
      document.write("<p>It's not the World.</p>");
    }
  </script>

</body>
```

</html>

This example demonstrates a basic HTML document structure with embedded JavaScript:

- **Variables:** Declaring variables using `let`. In this case, `greeting` and `name` are defined.
- **String Concatenation:** Concatenating strings using the `+` operator.
- **Alert:** Displaying an alert message using `alert()`.
- **Console Output:** Writing messages to the browser console using `console.log()`.
- **Conditional Statement:** Using an `if` statement to check if the value of `name` is equal to "World" and writing content to the document accordingly.

JavaScript Variables

Variables are Containers for Storing Data

JavaScript Variables can be declared in 4 ways:

- **Automatically**
- **Using** `var`
- **Using** `let`
- **Using** `const`

JavaScript Identifiers

All JavaScript variables must be identified with unique names.

These unique names are called identifiers.

Identifiers can be short names (like `x` and `y`) or more descriptive names (`age`, `sum`, `totalVolume`).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.

- Names can also begin with \$ and _ (but we will not use it in this tutorial).
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

In JavaScript, variables are used to store data values. You can declare variables using the `var`, `let`, or `const` keywords.

1.`var`:

Historically used for variable declaration in JavaScript, but its scope can be a bit tricky as it's function-scoped rather than block-scoped.

Example:

```
var x = 10;
```

2.`let`:

Introduced in ES6, `let` allows you to declare block-scoped variables, meaning they are limited to the block (enclosed by curly braces) in which they are defined or declared.

Example:

```
let y = 20;
```

3.`const`:

Also introduced in ES6, `const` is used to declare variables whose values should not be re-assigned. However, it doesn't mean the value itself is immutable; for example, you can still modify properties of objects declared with `const`.

Example:

```
const z = 30;
```

Variables can store various types of data including numbers, strings, booleans, arrays, objects, and functions. JavaScript is a dynamically-typed language, so you don't need to explicitly declare the type of a variable. The type of a variable is determined dynamically at runtime based on the type of value assigned to it.

Data Types In Javascript

JavaScript is a dynamically typed (also called loosely typed) scripting language. In JavaScript, variables can receive different data types over time. The latest ECMAScript standard defines eight data types Out of which seven data types are Primitive(predefined) and one complex or Non-Primitive.

Primitive Data Types

The predefined data types provided by JavaScript language are known as primitive data types. Primitive data types are also known as in-built data types.

- [Number](#): JavaScript numbers are always stored in double-precision 64-bit binary format IEEE 754. Unlike other programming languages, you don't need int, float, etc to declare different numeric values.
- [String](#): JavaScript Strings are similar to sentences. They are made up of a list of characters, which is essentially just an "array of characters, like "Hello GeeksforGeeks" etc.
- [Boolean](#): Represent a logical entity and can have two values: true or false.
- [Null](#): This type has only one value that is *null*.
- [Undefined](#): A variable that has not been assigned a value is *undefined*.
- [Symbol](#): Symbols return unique identifiers that can be used to add unique property keys to an object that won't collide with keys of any other code that might add to the object.

- [BigInt](#): BigInt is a built-in object in JavaScript that provides a way to represent whole numbers larger than 2⁵³-1.

Non-Primitive Data Types

The data types that are derived from primitive data types of the JavaScript language are known as non-primitive data types. It is also known as derived data types or reference data types.

- [Object](#): It is the most important data type and forms the building blocks for modern JavaScript. We will learn about these data types in detail in further articles.

JavaScript Operators

JavaScript operators operate the operands, these are symbols that are used to manipulate a certain value or operand. Operators are used to performing specific mathematical and logical computations on operands.

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators

JavaScript Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Arithmetic Operators are used to perform arithmetic on numbers:

JavaScript Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Assignment operators assign values to JavaScript variables.

The Addition Assignment Operator (**+=**) adds a value to a variable.

JavaScript Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JavaScript Logical Operators

Operator	Description
&&	logical and
	logical or
!	logical not

JavaScript Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

JavaScript if, else, and else if

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed

The if Statement

Use the **if** statement to specify a block of JavaScript code to be executed if a condition is true.

Syntax

```
if (condition) {
```

```
    // block of code to be executed if the condition is true
```

```
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript if</h2>
```

```
<p>Display "Good day!" if the hour is less than 18:00:</p>
```

```
<p id="demo">Good Evening!</p>
```

```
<script>
```

```
    if (new Date().getHours() < 18) {
```

```
        document.getElementById("demo").innerHTML = "Good  
        day!";
```

```
    }
```

```
</script>
```

```
</body>
```

```
</html>
```

The else Statement

Use the **else** statement to specify a block of code to be executed if the condition is false.

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>JavaScript if .. else</h2>
```

```
<p>A time-based greeting:</p>
```

```
<p id="demo"></p>
```

```
<script>  
    const hour = new Date().getHours();  
    let greeting;  
  
    if (hour < 18) {  
        greeting = "Good day";  
    } else {  
        greeting = "Good evening";  
    }  
  
    document.getElementById("demo").innerHTML = greeting;  
</script>  
  
</body>  
</html>
```

The else if Statement

Use the **else if** statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {  
  
    // block of code to be executed if condition1 is true  
  
} else if (condition2) {  
  
    // block of code to be executed if the condition1 is false and condition2 is true  
  
} else {  
  
    // block of code to be executed if the condition1 is false and condition2 is false  
  
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript if .. else</h2>
```

```
<p>A time-based greeting:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    const time = new Date().getHours();
```

```
    let greeting;
```

```
    if (time < 10) {
```

```
        greeting = "Good morning";
```

```
    } else if (time < 20) {
```

```
        greeting = "Good day";
```



```
        } else {  
            greeting = "Good evening";  
        }  
        document.getElementById("demo").innerHTML = greeting;  
</script>  
  
</body>  
</html>
```

The **switch** statement is used to perform different actions based on different conditions.

The JavaScript Switch Statement

Use the **switch** statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {  
  
    case x:  
  
        // code block  
  
        break;  
  
    case y:  
  
        // code block  
  
        break;  
  
    default:  
  
        // code block  
  
}
```

This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, the default code block is executed.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript switch</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    let day;
```

```
    switch (new Date().getDay()) {
```

```
        case 0:
```

```
            day = "Sunday";
```

```
            break;
```

```
        case 1:
```

```
            day = "Monday";
```

```
            break;
```

```
        case 2:
```

```
            day = "Tuesday";
```

```
            break;
```

```
        case 3:
```

```
            day = "Wednesday";
```

```
            break;
```

```
        case 4:
```

```
            day = "Thursday";
```

```
            break;
```

```
        case 5:
```

```
            day = "Friday";
```

```
            break;
```

```
        case 6:
```

```
            day = "Saturday";
```

```
    }
```

```
    document.getElementById("demo").innerHTML = "Today is " +  
    day;
```

</script>

</body>

</html>

The break Keyword

When JavaScript reaches a **break** keyword, it breaks out of the switch block.

This will stop the execution inside the switch block.

It is not necessary to break the last case in a switch block. The block breaks (ends) there anyway.

Note: If you omit the break statement, the next case will be executed even if the evaluation does not match the case.

JavaScript Loop

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Different Kinds of Loops

JavaScript supports different kinds of loops:

- **for** - loops through a block of code a number of times
- **for/in** - loops through the properties of an object
- **for/of** - loops through the values of an iterable object
- **while** - loops through a block of code while a specified condition is true
- **do/while** - also loops through a block of code while a specified condition is true

The For Loop

The **for** statement creates a loop with 3 optional expressions:

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

Expression 1 is executed (one time) before the execution of the code block.

Expression 2 defines the condition for executing the code block.

Expression 3 is executed (every time) after the code block has been executed.

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript For Loop</h2>  
  
<p id="demo"></p>  
  
<script>  
    let text = "";  
  
    for (let i = 0; i < 5; i++) {  
        text += "The number is " + i + "<br>";  
    }  
  
    document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

The For In Loop

The JavaScript **for in** statement loops through the properties of an Object:

Syntax

```
for (key in object) {
```

```
    // code block to be executed
```

```
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript For In Loop</h2>
```

```
<p>The for in statement loops through the properties of an object:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    const person = {fname:"John", lname:"Doe", age:25};
```

```
    let txt = "";
```

```
    for (let x in person) {
```

```
        txt += person[x] + " ";
```

```
    }
```

```
    document.getElementById("demo").innerHTML = txt;
```

```
</script>
```

```
</body>
```

```
</html>
```

The For Of Loop

The JavaScript **for of** statement loops through the values of an iterable object.

It lets you loop over iterable data structures such as Arrays, Strings, Maps, NodeLists, and more:

Syntax

```
for (variable of iterable) {
```

```
    // code block to be executed
```

```
}
```

variable - For every iteration the value of the next property is assigned to the variable. *Variable* can be declared with **const**, **let**, or **var**.

iterable - An object that has iterable properties.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript For Of Loop</h2>
```

```
<p>The for of statement loops through the values of any iterable object:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    const cars = ["BMW", "Volvo", "Mini"];
```

```
    let text = "";
```

```
    for (let x of cars) {
```

```
        text += x + "<br>";
```

```
    }

    document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

The While Loop

The **while** loop loops through a block of code as long as a specified condition is true.

Syntax

```
while (condition) {

    // code block to be executed

}
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript While Loop</h2>

<p id="demo"></p>

<script>
    let text = "";
    let i = 0;
    while (i < 10) {
        text += "<br>The number is " + i;
        i++;
    }
    document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

The Do While Loop

The **do while** loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {
    // code block to be executed
}
```

```
while (condition);
```

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript Do While Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
    let text = ""
    let i = 0;

    do {
        text += "<br>The number is " + i;
        i++;
    }
    while (i < 10);

    document.getElementById("demo").innerHTML = text;
</script>
```



```
</body>
</html>
```

The Break Statement

You have already seen the **break** statement used in an earlier chapter of this tutorial. It was used to "jump out" of a **switch()** statement.

The **break** statement can also be used to jump out of a loop:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Loops</h2>

<p>A loop with a <b>break</b> statement.</p>

<p id="demo"></p>

<script>
    let text = "";
    for (let i = 0; i < 10; i++) {
        if (i === 3) { break; }
        text += "The number is " + i + "<br>";
    }

    document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

The Continue Statement

The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 3:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Loops</h2>

<p>A loop with a <b>continue</b> statement.</p>

<p>A loop which will skip the step where i = 3.</p>

<p id="demo"></p>

<script>
    let text = "";
    for (let i = 0; i < 10; i++) {
        if (i === 3) { continue; }
        text += "The number is " + i + "<br>";
    }
    document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript Functions

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

Syntax

A JavaScript function is defined with the **function** keyword, followed by a name, followed by parentheses ().

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas: (*parameter1, parameter2, ...*)

The code to be executed, by the function, is placed inside curly brackets: {}

```
function name(parameter1, parameter2, parameter3) {  
  
    // code to be executed  
  
}
```

Function parameters are listed inside the parentheses () in the function definition.

Function arguments are the values received by the function when it is invoked.

Inside the function, the arguments (the parameters) behave as local variables.

Function Invocation

The code inside the function will execute when "something" invokes (calls) the function:

- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code
- Automatically (self invoked)

Function Return

When JavaScript reaches a **return** statement, the function will stop executing.

If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>
    let x = myFunction(15,5);
    document.getElementById("demo").innerHTML = x;

    function myFunction(a, b) {
        return a + b;
    }
</script>

</body>
</html>
```

Why Functions?

With functions you can reuse code

You can write code that can be used many times.

You can use the same code with different arguments, to produce different results.

Function Parameters and Arguments

```
function functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

Function parameters are the names listed in the function definition.

Function arguments are the real values passed to (and received by) the function.

Parameter Rules

JavaScript function definitions do not specify data types for parameters.

JavaScript functions do not perform type checking on the passed arguments.

JavaScript functions do not check the number of arguments received.

Default Parameters

If a function is called with missing arguments (less than declared), the missing values are set to undefined.

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<p>Setting a default value to a function parameter.</p>  
<p id="demo"></p>
```

```
<script>  
    function myFunction(x, y) {
```

```
        if (y === undefined) {
            y = 2;
        }
        return x * y;
    }
    document.getElementById("demo").innerHTML = myFunction(4);
</script>

</body>
</html>
```

Default Parameter Values

ES6 allows function parameters to have default values.

Example

If y is not passed or undefined, then y = 10.

```
function myFunction(x, y = 10) {

    return x + y;

}

myFunction(5);
```

Function Rest Parameter

The rest parameter (...) allows a function to treat an indefinite number of arguments as an array:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<h2>The Rest Parameter</h2>
```

<p>The rest parameter (...) allows a function to treat an indefinite number of arguments as an array:</p>

```
<p id="demo"></p>

<script>
    function sum(...args) {
        let sum = 0;
        for (let arg of args) sum += arg;
        return sum;
    }

    let x = sum(4, 9, 16, 25, 29, 100, 66, 77);

    document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

Invoking a JavaScript Function

The code inside a function is not executed when the function is defined.

The code inside a function is executed when the function is invoked.

It is common to use the term "call a function" instead of "invoke a function".

It is also common to say "call upon a function", "start a function", or "execute a function".

In this tutorial, we will use `invoke`, because a JavaScript function can be invoked without being called.

Invoking a Function as a Function

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>

<p>The global function (myFunction) returns the product of the arguments
(a,b):</p>

<p id="demo"></p>

<script>
function myFunction(a, b) {
  return a * b;
}
document.getElementById("demo").innerHTML = myFunction(10, 2);
</script>

</body>
</html>
```

Invoking a Function as a Method

In JavaScript you can define functions as object methods.

The following example creates an object (myObject), with two properties (firstName and lastName), and a method (fullName):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>

<p>myObject.fullName() will return John Doe:</p>

<p id="demo"></p>
```



```
<script>
const myObject = {
  firstName:"John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
document.getElementById("demo").innerHTML = myObject.fullName();
</script>

</body>
</html>
```

Invoking a Function with a Function Constructor

If a function invocation is preceded with the new keyword, it is a constructor invocation.

It looks like you create a new function, but since JavaScript functions are objects you actually create a new object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>In this example, myFunction is a function constructor:</p>

<p id="demo"></p>

<script>
function myFunction(arg1, arg2) {
  this.firstName = arg1;
  this.lastName = arg2;
```

```
}  
  
const myObj = new myFunction("John","Doe")  
document.getElementById("demo").innerHTML = myObj.firstName;  
</script>  
  
</body>  
</html>
```

What is this?

In JavaScript, this keyword refers to an object.

Which object depends on how this is being invoked (used or called).

The this keyword refers to different objects depending on how it is used:

- In an object method, this refers to the object.
- Alone, this refers to the global object.
- In a function, this refers to the global object.
- In a function, in strict mode, this is undefined.
- In an event, this refers to the element that received the event.
- Methods like call(), apply(), and bind() can refer this to any object.

The JavaScript call() Method

The call() method is a predefined JavaScript method.

It can be used to invoke (call) a method with an owner object as an argument (parameter).

With call(), an object can use a method belonging to another object.

This example calls the fullName method of person, using it on person1:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Functions</h2>  
<p>This example calls the fullName method of person, using it on person1:
```

</p>

<p id="demo"></p>

<script>

```
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
const person1 = {
  firstName:"John",
  lastName: "Doe"
}
const person2 = {
  firstName:"Mary",
  lastName: "Doe"
}
document.getElementById("demo").innerHTML =
person.fullName.call(person1);
</script>
```

</body>

</html>

The call() Method with Arguments

The call() method can accept arguments:

<!DOCTYPE html>

<html>

<body>

<h2>JavaScript Functions</h2>

<p>This example calls the fullName method of person, using it on person1:
</p>

<p id="demo"></p>

<script>

```
const person = {
```

```

    fullName: function(city, country) {
        return this.firstName + " " + this.lastName + "," + city + "," + country;
    }
}

const person1 = {
    firstName:"John",
    lastName: "Doe"
}

const person2 = {
    firstName:"Mary",
    lastName: "Doe"
}

document.getElementById("demo").innerHTML =
person.fullName.call(person1, "Oslo", "Norway");
</script>

</body>
</html>

```

The JavaScript apply() Method

The apply() method is similar to the call() method (previous chapter).

In this example the fullName method of person is applied on person1:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>In this example the fullName method of person is <b>applied</b> on
person1:</p>

<p id="demo"></p>

<script>
const person = {
    fullName: function() {

```

```
        return this.firstName + " " + this.lastName;
    }
}

const person1 = {
    firstName:"John",
    lastName: "Doe"
}

document.getElementById("demo").innerHTML =
person.fullName.apply(person1);
</script>

</body>
</html>
```

The Difference Between call() and apply()

The difference is:

The call() method takes arguments separately.

The apply() method takes arguments as an array.

The apply() method is very handy if you want to use an array instead of an argument list.

The apply() Method with Arguments

The apply() method accepts arguments in an array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>In this example the fullName method of person is <b>applied</b> on
person1:</p>
```

```
<p id="demo"></p>
```

```
<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + "," + city + "," + country;
  }
}

const person1 = {
  firstName:"John",
  lastName: "Doe"
}

document.getElementById("demo").innerHTML =
person.fullName.apply(person1, ["Oslo", "Norway"]);
</script>

</body>
</html>
```

Simulate a Max Method on Arrays

You can find the largest number (in a list of numbers) using the Math.max() method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Math.max()</h2>
<p>This example returns the highest number in a list of number
arguments:</p>

<p id="demo"></p>
```

```
<script>
document.getElementById("demo").innerHTML = Math.max(1,2,3);
</script>

</body>
</html>
```

JavaScript Function bind()

Function Borrowing

With the bind() method, an object can borrow a method from another object.

The example below creates 2 objects (person and member).

The member object borrows the fullname method from the person object:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Function bind()</h1>

<p>This example creates 2 objects (person and member).</p>
<p>The member object borrows the fullname method from person:</p>

<p id="demo"></p>

<script>
const person = {
  firstName:"John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}

const member = {
```

```
    firstName:"Hege",  
    lastName: "Nilsen",  
  }
```

```
let fullName = person.fullName.bind(member);
```

```
document.getElementById("demo").innerHTML = fullName();  
</script>
```

```
</body>  
</html>
```

JavaScript Closures

JavaScript variables can belong to the local or global scope.

Global variables can be made local (private) with closures.

Global Variables

A function can access all variables defined inside the function, like this:

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>JavaScript Functions</h2>
```

```
<p>A function can access variables defined inside the function:</p>
```

```
<p id="demo"></p>
```

```
<script>  
myFunction();
```

```
function myFunction() {  
  let a = 4;  
  document.getElementById("demo").innerHTML = a * a;  
}
```



```
</script>
```

```
</body>
```

```
</html>
```

JavaScript Higher Order Functions

In JavaScript, a higher-order function is a function that can accept other functions as arguments, return functions, or both. They enable abstraction, composition, and the creation of more flexible and reusable code.

Syntax:

```
function higherOrderFunction(callback) {
```

```
    // Perform some operations
```

```
    // Call the callback function
```

```
    callback();
```

```
}
```

```
function callbackFunction() {
```

```
    console.log("Callback function is executed.");
```

```
}
```

```
// Passing the callback function to the higher-order function
```

```
higherOrderFunction(callbackFunction);
```

Parameters:

higherOrderFunction: Takes a callback function, executes it, and performs operations.

callback: A function passed as an argument, executed inside higherOrderFunction.

callbackFunction(): Logs “Callback function is executed.”

Invocation: Calls higherOrderFunction(callbackFunction), executing callbackFunction within higherOrderFunction.

Approach 1: Function as an Argument:

This approach involves passing a function (callback) as an argument to another function. The receiving function can then execute the callback, enabling flexible and customizable behavior in JavaScript programs.

Example: In this example, Two functions: greet accepts name and returns a greeting. greet_name combines greeting, message, and name, logging a customized message. Output: “Hi!! JavaScript Welcome To Computer Vision.

```
function greet(name) {  
  
    return `Hi!! ${name} `;  
  
}  
  
function greet_name(greeting, message, name) {  
  
    console.log(` ${greeting(name)} ${message} `);  
  
}  
  
greet_name(greet, 'Welcome To Computer Vision', 'Comp');
```

Approach 2 : Functions as Return Values:

Higher-order functions can also return new functions. This is often used for creating specialized functions or closures. For instance, you can create a function factory that generates functions with specific behavior.

Example: In this example, `multiplier` is a higher-order function that takes a `factor` as an argument and returns a new function that multiplies any number by that factor. We then use `double` and `triple` to create specialized functions.

```
function multiplier(factor) {  
    return function (x) {  
        return x * factor;  
    };  
}  
  
const double = multiplier(2);  
  
const triple = multiplier(3);  
  
console.log(double(5));  
  
console.log(triple(5));
```

Arrays:

Arrays in JavaScript are ordered collections of values. Unlike objects, which use named keys, arrays use numbered indexes to access elements. Arrays can hold elements of any data type, including other arrays and objects, making them versatile for storing and manipulating collections of data.

Creating Arrays:

You can create arrays using array literals:

```
const fruits = ['apple', 'banana', 'orange'];
```

Why Use Arrays?

If you have a list of items (a list of car names, for example), storing the courses in single variables could look like this:

```
let course1 = "C";
```

```
let course2 = "C++";
```

```
let courses3 = "PYTHON";
```

However, what if you want to loop through the courses and find a specific one? And what if you had not 3 courses, but 300?

The solution is an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Arrays</h1>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const courses = ["C", "C++", "PYTHON"];
```

```
document.getElementById("demo").innerHTML = courses;
```

```
</script>
```

```
</body>
```

```
</html>
```

Using the JavaScript Keyword new

The following example also creates an Array, and assigns values to it:

```
const courses = new Array("C", "C++", "PYTHON");
```

Accessing Array Elements

You access an array element by referring to the index number:

```
const courses = ["C", "C++", "PYTHON"];
```

```
let course = courses[0];
```

Looping Array Elements

One way to loop through an array, is using a for loop:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Arrays</h1>
```

```
<h2>Looping an Array</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let fLen = fruits.length;
```

```
let text = "<ul>";
```

```
for (let i = 0; i < fLen; i++) {
```

```
    text += "<li>" + fruits[i] + "</li>";
```

```
}
```

```
text += "</ul>";
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

You can also use the `Array.forEach()` function:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Arrays</h1>
```

```
<h2>The forEach() Method</h2>
```

```
<p>Call a function for each array element:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let text = "<ul>";
```

```
fruits.forEach(myFunction);
```

```
text += "</ul>";
```

```
document.getElementById("demo").innerHTML = text;
```

```
function myFunction(value) {
```

```
    text += "<li>" + value + "</li>";
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```


JavaScript Array Methods

JavaScript Array length

The length property returns the length (size) of an array:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let size = fruits.length;
```

JavaScript Array toString()

The JavaScript method toString() converts an array to a string of (comma separated) array values.

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

document.getElementById("demo").innerHTML = fruits.toString();
```

Result:

Banana,Orange,Apple,Mango

JavaScript Array at()

ES2022 introduced the array method `at()`:

Examples

Get the third element of fruits using `at()`:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let fruit = fruits.at(2);
```

Get the third element of fruits using `[]`:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let fruit = fruits[2];
```

The `at()` method returns an indexed element from an array.

The `at()` method returns the same as `[]`.

The `at()` method is supported in all modern browsers since March 2022:

JavaScript Array join()

The `join()` method also joins all array elements into a string.

It behaves just like `toString()`, but in addition you can specify the separator:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

document.getElementById("demo").innerHTML = fruits.join(" * ");
```

Result:

Banana * Orange * Apple * Mango

Popping and Pushing

When you work with arrays, it is easy to remove elements and add new elements.

This is what popping and pushing is:

Popping items out of an array, or pushing items into an array.

JavaScript Array `pop()`

The `pop()` method removes the last element from an array:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.pop();
```

The `pop()` method returns the value that was "popped out":

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let fruit = fruits.pop();
```

JavaScript Array `push()`

The `push()` method adds a new element to an array (at the end):

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.push("Kiwi");
```

The `push()` method returns the new array length:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let length = fruits.push("Kiwi");
```

Shifting Elements

Shifting is equivalent to popping, but working on the first element instead of the last.

JavaScript Array shift()

The shift() method removes the first array element and "shifts" all other elements to a lower index.

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.shift();
```

The shift() method returns the value that was "shifted out":

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let fruit = fruits.shift();
```

JavaScript Array unshift()

The unshift() method adds a new element to an array (at the beginning), and "unshifts" older elements:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.unshift("Lemon");
```

The unshift() method returns the new array length:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.unshift("Lemon");
```

Changing Elements

Array elements are accessed using their index number:

Array indexes start with 0:

[0] is the first array element

[1] is the second

[2] is the third ...

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits[0] = "Kiwi";
```

JavaScript Array length

The length property provides an easy way to append a new element to an array:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits[fruits.length] = "Kiwi";
```

JavaScript Array delete()

Warning !

Using delete() leaves undefined holes in the array.

Use `pop()` or `shift()` instead.

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
delete fruits[0];
```

Merging Arrays (Concatenating)

In programming languages, concatenation means joining strings end-to-end.

Concatenation "snow" and "ball" gives "snowball".

Concatenating arrays means joining arrays end-to-end.

JavaScript Array `concat()`

The `concat()` method creates a new array by merging (concatenating) existing arrays:

Example (Merging Two Arrays)

```
const myGirls = ["Cecilie", "Lone"];
```

```
const myBoys = ["Emil", "Tobias", "Linus"];
```

```
const myChildren = myGirls.concat(myBoys);
```

Note

The `concat()` method does not change the existing arrays. It always returns a new array.

The `concat()` method can take any number of array arguments.

Example (Merging Three Arrays)

```
const arr1 = ["Cecilie", "Lone"];
const arr2 = ["Emil", "Tobias", "Linus"];
const arr3 = ["Robin", "Morgan"];
const myChildren = arr1.concat(arr2, arr3);
```

The `concat()` method can also take strings as arguments:

Example (Merging an Array with Values)

```
const arr1 = ["Emil", "Tobias", "Linus"];
const myChildren = arr1.concat("Peter");
```

Splicing and Slicing Arrays

The `splice()` method adds new items to an array.

The `slice()` method slices out a piece of an array.

JavaScript Array `splice()`

The `splice()` method can be used to add new items to an array:

Example


```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.splice(2, 0, "Lemon", "Kiwi");
```

The first parameter (2) defines the position where new elements should be added (spliced in).

The second parameter (0) defines how many elements should be removed.

The rest of the parameters ("Lemon" , "Kiwi") define the new elements to be added.

The splice() method returns an array with the deleted items:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.splice(2, 2, "Lemon", "Kiwi");
```

Using splice() to Remove Elements

With clever parameter setting, you can use splice() to remove elements without leaving "holes" in the array:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.splice(0, 1);
```

The first parameter (0) defines the position where new elements should be added (spliced in).

The second parameter (1) defines how many elements should be removed.

The rest of the parameters are omitted. No new elements will be added.

JavaScript Objects

JavaScript, an object is a fundamental data type that represents a collection of key-value pairs. It's a versatile and powerful construct used for organizing and manipulating data. Objects allow you to store various types of data, including other objects, arrays, functions, and primitive values like strings, numbers, and booleans.

In JavaScript, almost "everything" is an object.

- Booleans can be objects (if defined with the **new** keyword)
- Numbers can be objects (if defined with the **new** keyword)
- Strings can be objects (if defined with the **new** keyword)
- Dates are always objects
- Maths are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are always objects

Example:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating an object:</p>

<p id="demo"></p>

<script>
const person = {
  firstName : "John",
  lastName  : "Doe",
  age       : 50,
  eyeColor  : "blue"
};
```

```
document.getElementById("demo").innerHTML = person.firstName
+ " " + person.lastName;
</script>

</body>
</html>
```

Creating a JavaScript Object

With JavaScript, you can define and create your own objects.

There are different ways to create new objects:

- Create a single object, using an object literal.
- Create a single object, with the keyword **new**.
- Define an object constructor, and then create objects of the constructed type.
- Create an object using **Object.create()**.

Using an Object Literal

This is the easiest way to create a JavaScript Object.

Using an object literal, you both define and create an object in one statement.

An object literal is a list of name:value pairs (like age:50) inside curly braces {}.

The following example creates a new JavaScript object with four properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating a JavaScript Object:</p>

<p id="demo"></p>
```

```
<script>
const person = {firstName:"John", lastName:"Doe",
age:50,eyeColor:"blue"};

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

Using the JavaScript Keyword new

The following example create a new JavaScript object using **new Object()**, and then adds 4 properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating a JavaScript Object:</p>

<p id="demo"></p>

<script>
const person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

JavaScript Object Properties

Properties are the values associated with a JavaScript object.

A JavaScript object is a collection of unordered properties.

Properties can usually be changed, added, and deleted, but some are read only.

Accessing JavaScript Properties

The syntax for accessing the property of an object is:

```
objectName.property      // person.age
```

or

```
objectName["property"]   // person["age"]
```

or

```
objectName[expression]   // x = "age"; person[x]
```

JavaScript for...in Loop

The JavaScript **for...in** statement loops through the properties of an object.

Syntax

```
for (let variable in object) {  
    // code to be executed  
}
```

The block of code inside of the **for...in** loop will be executed once for each property.

Looping through the properties of an object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>Looping object property values:</p>

<p id="demo"></p>

<script>
const person = {
  fname:"John",
  lname:"Doe",
  age:25
};

let txt = "";
for (let x in person) {
  txt += person[x] + " ";
}

document.getElementById("demo").innerHTML = txt;
</script>

</body>
</html>
```

Adding New Properties

You can add new properties to an existing object by simply giving it a value.

Assume that the person object already exists - you can then give it new properties:

```
person.nationality = "English";
```

Deleting Properties

The **delete** keyword deletes a property from an object:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};  
  
delete person.age;
```

The **delete** keyword deletes both the value of the property and the property itself.

After deletion, the property cannot be used before it is added back again.

The **delete** operator is designed to be used on object properties. It has no effect on variables or functions.

The **delete** operator should not be used on predefined JavaScript object properties. It can crash your application.

Nested Objects

Values in an object can be another object:

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>JavaScript Objects</h2>  
<p>Access nested objects:</p>  
  
<p id="demo"></p>  
  
<script>
```

```

const myObj = {
  name: "John",
  age: 30,
  cars: {
    car1: "Ford",
    car2: "BMW",
    car3: "Fiat"
  }
}
document.getElementById("demo").innerHTML = myObj.cars.car2;
</script>

</body>
</html>

```

Nested Arrays and Objects

Values in objects can be arrays, and values in arrays can be objects:

```

<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>Nested JavaScript Objects and Arrays.</h2>

<p id="demo"></p>

<script>
let x = "";
const myObj = {
  name: "John",
  age: 30,
  cars: [
    {name:"Ford", models:["Fiesta", "Focus", "Mustang"]},
    {name:"BMW", models:["320", "X3", "X5"]},
    {name:"Fiat", models:["500", "Panda"]}
  ]
}

for (let i in myObj.cars) {
  x += "<h2>" + myObj.cars[i].name + "</h2>";
  for (let j in myObj.cars[i].models) {
    x += myObj.cars[i].models[j] + "<br>";
  }
}

```



```

    }
}

document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>

```

JavaScript Object Methods

```

<!DOCTYPE html>
<html>
<body>

<h1>The JavaScript <i>this</i> Keyword</h1>
<p>In this example, <b>this</b> refers to the <b>person</b>
object.</p>
<p>Because <b>fullName</b> is a method of the person object.</p>

<p id="demo"></p>

<script>
// Create an object:
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};

// Display data from the object:
document.getElementById("demo").innerHTML = person.fullName();
</script>

</body>
</html>

```

What is this?

In JavaScript, the `this` keyword refers to an object.

Which object depends on how `this` is being invoked (used or called).

The `this` keyword refers to different objects depending on how it is used:

In an object method, `this` refers to the object.

Alone, `this` refers to the global object.

In a function, `this` refers to the global object.

In a function, in strict mode, `this` is `undefined`.

In an event, `this` refers to the element that received the event.

Methods like `call()`, `apply()`, and `bind()` can refer `this` to any object.

Accessing Object Methods

You access an object method with the following syntax:

```
objectName.methodName()
```

You will typically describe `fullName()` as a method of the person object, and `fullName` as a property.

The `fullName` property will execute (as a function) when it is invoked with `()`.

This example accesses the `fullName()` method of a person object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating and using an object method.</p>
<p>A method is actually a function definition stored as a
property value.</p>

<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>
</body>
</html>
```

Using Built-In Methods

This example uses the `toUpperCase()` method of the String object, to convert a text to uppercase:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript Objects</h2>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
};
person.name = function() {
  return (this.firstName + " " + this.lastName).toUpperCase();
};

document.getElementById("demo").innerHTML =
"My father is " + person.name();
</script>

</body>
</html>
```

JavaScript Display Objects

How to Display JavaScript Objects?

Displaying a JavaScript object will output [object Object].

Some common solutions to display JavaScript objects are:

- Displaying the Object Properties by name
- Displaying the Object Properties in a Loop
- Displaying the Object using Object.values()
- Displaying the Object using JSON.stringify()

Displaying Object Properties

The properties of an object can be displayed as a string:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Objects</h2>
```

```
<p>Display object properties:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const person = {
```

```
  name: "John",
```

```
  age: 30,
```

```
  city: "New York"
```

```
};
```

```
document.getElementById("demo").innerHTML = person.name + ", "  
+ person.age + ", " + person.city;
```

```
</script>
```

```
</body>
```

```
</html>
```

Displaying the Object in a Loop

The properties of an object can be collected in a loop:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Objects</h2>
```

```
<p>Display object properties:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const person = {
```

```
  name: "John",
```

```
  age: 30,
```

```
  city: "New York"
```

```
};
```

```
let txt = "";
```

```
for (let x in person) {
```

```
  txt += person[x] + " ";
```

```
};
```

```
document.getElementById("demo").innerHTML = txt;
```

```
</script>
```

```
</body>
```

```
</html>
```

Using Object.values()

Any JavaScript object can be converted to an array using `Object.values()`:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Objects</h2>
```

```
<p>Object.values() converts an object to an array.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const person = {
```

```
  name: "John",
```

```
  age: 30,
```

```
  city: "New York"
```

```
};
```

```
document.getElementById("demo").innerHTML =  
Object.values(person);
```

```
</script>
```

```
</body>
```

```
</html>
```

Using JSON.stringify()

Any JavaScript object can be stringified (converted to a string) with the JavaScript function `JSON.stringify()`:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Objects</h2>
```

```
<p>Display properties in JSON format:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const person = {
```

```
  name: "John",
```

```
  age: 30,
```

```
  city: "New York"
```

```
};
```



```
document.getElementById("demo").innerHTML =  
JSON.stringify(person);
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript Accessors (Getters and Setters)

ECMAScript 5 (ES5 2009) introduced Getter and Setters.

Getters and setters allow you to define Object Accessors (Computed Properties).

JavaScript Getter (The get Keyword)

This example uses a **lang** property to **get** the value of the **language** property.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Getters and Setters</h2>
```

```
<p>Getters and setters allow you to get and set object properties via  
methods.</p>
```

<p>This example uses a lang property to get the value of the language property:</p>

<p id="demo"></p>

<script>

// Create an object:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "en",  
  get lang() {  
    return this.language;  
  }  
};
```

// Display data from the object using a getter:

```
document.getElementById("demo").innerHTML = person.lang;
```

</script>

</body>

</html>

JavaScript Setter (The set Keyword)

This example uses a **lang** property to **set** the value of the **language** property.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Getters and Setters</h2>
```

```
<p>Getters and setters allow you to get and set properties via methods.</p>
```

```
<p>This example uses a lang property to set the value of the language property.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
// Create an object:
```

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "NO",  
  set lang(value) {  
    this.language = value;  
  }  
}
```

```
};
```

```
// Set a property using set:
```

```
person.lang = "en";
```

```
// Display data from the object:
```

```
document.getElementById("demo").innerHTML = person.language;
```

```
</script>
```

```
</body>
```

```
</html>
```

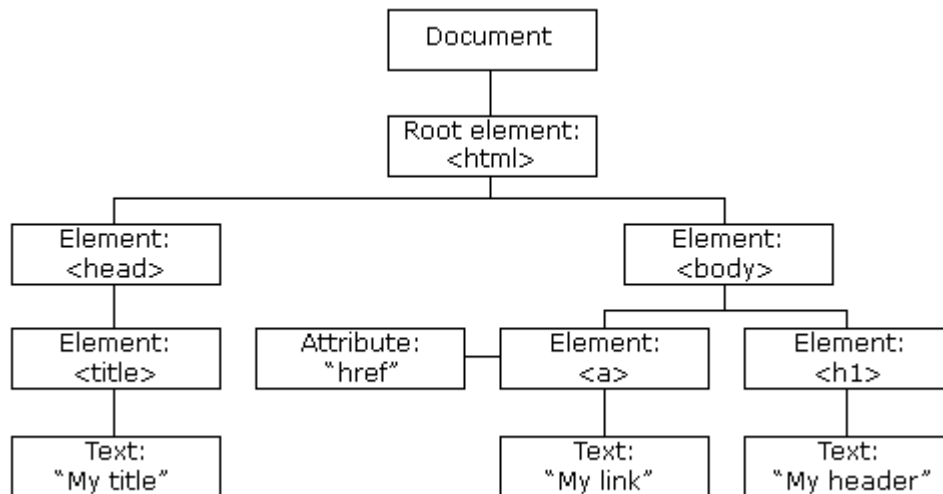
Why Using Getters and Setters?

- **It gives simpler syntax**
- **It allows equal syntax for properties and methods**
- **It can secure better data quality**
- **It is useful for doing things behind-the-scenes**

JavaScript HTML DOM

With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

The HTML DOM Tree of Objects



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

What is the DOM?

The DOM is a W3C (World Wide Web Consortium) standard.

The DOM defines a standard for accessing documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The W3C DOM standard is separated into 3 different parts:

- **Core DOM** - standard model for all document types
 - **XML DOM** - standard model for XML documents
 - **HTML DOM** - standard model for HTML documents
-

What is the HTML DOM?

The HTML DOM is a standard object model and programming interface for HTML. It defines:

- **The HTML elements as objects**
- **The properties of all HTML elements**
- **The methods to access all HTML elements**
- **The events for all HTML elements**

In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

JavaScript - HTML DOM Methods

HTML DOM methods are actions you can perform (on HTML Elements).

HTML DOM properties are values (of HTML Elements) that you can set or change.

The DOM Programming Interface

The HTML DOM can be accessed with JavaScript (and with other programming languages).

In the DOM, all HTML elements are defined as objects.

The programming interface is the properties and methods of each object.

A property is a value that you can get or set (like changing the content of an HTML element).

A method is an action you can do (like add or deleting an HTML element).

Example

The following example changes the content (the `innerHTML`) of the `<p>` element with `id="demo"`:

```
<html>

<body>

<p id="demo"></p>

<script>

document.getElementById("demo").innerHTML = "Hello World!";

</script>

</body>

</html>
```

In the example above, `getElementById` is a method, while `innerHTML` is a property.

The getElementById Method

The most common way to access an HTML element is to use the `id` of the element.

In the example above the `getElementById` method used `id="demo"` to find the element.

The innerHTML Property

The easiest way to get the content of an element is by using the `innerHTML` property.

The `innerHTML` property is useful for getting or replacing the content of HTML elements.

JavaScript HTML DOM Document

The HTML DOM document object is the owner of all other objects in your web page.

The HTML DOM Document Object

The document object represents your web page.

If you want to access any element in an HTML page, you always start with accessing the document object.

Below are some examples of how you can use the document object to access and manipulate HTML.

Finding HTML Elements

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Changing HTML Elements

Property	Description
<code>element.innerHTML = <i>new html content</i></code>	Change the inner HTML of an element
<code>element.attribute = <i>new value</i></code>	Change the attribute value of an HTML element
<code>element.style.property = <i>new style</i></code>	Change the style of an HTML element
Method	Description
<code>element.setAttribute(<i>attribute</i>, <i>value</i>)</code>	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Adding Events Handle

Method	Description
<code>document.getElementById(id).onclick = function(){code}</code>	Adding event handler code to an onclick event

JavaScript HTML DOM Elements

This page teaches you how to find and access HTML elements in an HTML page.

Finding HTML Elements

Often, with JavaScript, you want to manipulate HTML elements.

To do so, you have to find the elements first. There are several ways to do this:

- Finding HTML elements by id
- Finding HTML elements by tag name
- Finding HTML elements by class name
- Finding HTML elements by CSS selectors
- Finding HTML elements by HTML object collections

Finding HTML Element by Id

The easiest way to find an HTML element in the DOM, is by using the element id.

This example finds the element with `id="intro"`:

```
const element = document.getElementById("intro");
```

Finding HTML Elements by Tag Name

This example finds all `<p>` elements:

```
const element = document.getElementsByTagName("p");
```

Finding HTML Elements by Class Name

If you want to find all HTML elements with the same class name, use `getElementsByClassName()`.

This example returns a list of all elements with `class="intro"`.

```
const x = document.getElementsByClassName("intro");
```

Finding HTML Elements by CSS Selectors

If you want to find all HTML elements that match a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.

This example returns a list of all `<p>` elements with `class="intro"`.

```
const x = document.querySelectorAll("p.intro");
```

Finding HTML Elements by HTML Object Collections

This example finds the form element with `id="frm1"`, in the forms collection, and displays all element values:

```
<!DOCTYPE html>

<html>

<body>


<h2>JavaScript HTML DOM</h2>

<p>Finding HTML Elements Using <b>document.forms</b>.</p>


<form id="frm1" action="/action_page.php">

    First name: <input type="text" name="fname" value="Donald"><br>

    Last name: <input type="text" name="lname"
value="Duck"><br><br>

    <input type="submit" value="Submit">

</form>

<p>These are the values of each element in the form:</p>

<p id="demo"></p>

<script>

const x = document.forms["frm1"];

let text = "";

for (let i = 0; i < x.length ;i++) {

    text += x.elements[i].value + "<br>";

}

document.getElementById("demo").innerHTML = text;

</script>


</body>

</html>
```

The following HTML objects (and object collections) are also accessible:

- [document.anchors](#)
- [document.body](#)
- [document.documentElement](#)
- [document.embeds](#)
- [document.forms](#)
- [document.head](#)
- [document.images](#)
- [document.links](#)
- [document.scripts](#)
- [document.title](#)

JavaScript HTML DOM - Changing HTML

Changing HTML Content

The easiest way to modify the content of an HTML element is by using the `innerHTML` property.

To change the content of an HTML element, use this syntax:

```
document.getElementById(id).innerHTML = new HTML
```

This example changes the content of a `<p>` element:

```
<html>
```

```
<body>
```

```
<p id="p1">Hello World!</p>
```

```
<script>
```

```
document.getElementById("p1").innerHTML = "New text!";
```

```
</script>
```

```
</body>
</html>
```

Changing the Value of an Attribute

To change the value of an HTML attribute, use this syntax:

```
document.getElementById(id).attribute = new value
```

This example changes the value of the src attribute of an `<img>` element:

```
<!DOCTYPE html>
<html>
<body>



<script>
document.getElementById("myImage").src = "landscape.jpg";
</script>

</body>
</html>
```

Dynamic HTML content

JavaScript can create dynamic HTML content:

Date : Wed Mar 20 2024 14:16:33 GMT+0530 (India Standard Time)

```
<!DOCTYPE html>
<html>
<body>
```

```
<script>
document.getElementById("demo").innerHTML = "Date : " +
Date(); </script>

</body>
</html>
```

document.write()

In JavaScript, `document.write()` can be used to write directly to the HTML output stream:

```
<!DOCTYPE html>
<html>
<body>

<p>Bla bla bla</p>

<script>
document.write(Date());
</script>

<p>Bla bla bla</p>

</body>
</html>
```

JavaScript Forms

JavaScript Form Validation

HTML form validation can be done by JavaScript.

If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

```
<!DOCTYPE html>

<html>

<head>

<script>

function validateForm() {

    let x = document.forms["myForm"]["fname"].value;

    if (x == "") {

        alert("Name must be filled out");

        return false;

    }

}

</script>

</head>

<body>
```



```
<h2>JavaScript Validation</h2>
```

```
<form name="myForm" action="/action_page.php" onsubmit="return  
validateForm()" method="post">
```

```
  Name: <input type="text" name="fname">
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

Automatic HTML Form Validation

HTML form validation can be performed automatically by the browser:

If a form field (fname) is empty, the `required` attribute prevents this form from being submitted:

```
<form action="/action_page.php" method="post">
```

```
  <input type="text" name="fname" required>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

Data Validation

Data validation is the process of ensuring that user input is clean, correct, and useful.

Typical validation tasks are:

- has the user filled in all required fields?
- has the user entered a valid date?
- has the user entered text in a numeric field?

Most often, the purpose of data validation is to ensure correct user input.

Validation can be defined by many different methods, and deployed in many different ways.

Server side validation is performed by a web server, after input has been sent to the server.

Client side validation is performed by a web browser, before input is sent to a web server.

HTML Constraint Validation

HTML5 introduced a new HTML validation concept called constraint validation.

HTML constraint validation is based on:

- Constraint validation HTML Input Attributes
- Constraint validation CSS Pseudo Selectors
- Constraint validation DOM Properties and Methods

Constraint Validation HTML Input Attributes

Attribute	Description
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

Selectors

Selector	Description
:disabled	Selects input elements with the "disabled" attribute specified

:invalid	Selects input elements with invalid values
:optional	Selects input elements with no "required" attribute specified
:required	Selects input elements with the "required" attribute specified
:valid	Selects input elements with valid values

JavaScript HTML DOM - Changing CSS

Changing HTML Style

To change the style of an HTML element, use this syntax:

```
document.getElementById(id).style.property = new style
```

The following example changes the style of a `<p>` element:

```
<html>
<body>

<p id="p2">Hello World!</p>

<script>
document.getElementById("p2").style.color = "blue";
</script>

</body>
</html>
```

Using Events

The HTML DOM allows you to execute code when an event occurs.

Events are generated by the browser when "things happen" to HTML elements:

- An element is clicked on
- The page has loaded
- Input fields are changed

You will learn more about events in the next chapter of this tutorial.

This example changes the style of the HTML element with `id="id1"`, when the user clicks a button:

```
<!DOCTYPE html>
<html>
<body>

<h1 id="id1">My Heading 1</h1>

<button type="button"
onclick="document.getElementById('id1').style.color =
'red'">
Click Me!</button>

</body>
</html>
```

JavaScript HTML DOM Events

Reacting to Events

A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.

To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:

`onclick=JavaScript`

Examples of HTML events:

- When a user clicks the mouse
- When a web page has loaded
- When an image has been loaded
- When the mouse moves over an element
- When an input field is changed
- When an HTML form is submitted
- When a user strokes a key

In this example, the content of the `<h1>` element is changed when a user clicks on it:

```
<!DOCTYPE html>
<html>
<body>

<h1 onclick="this.innerHTML = 'Oops!'">Click on this
text!</h1>

</body>
</html>
```

JavaScript HTML DOM EventListener

Add an event listener that fires when a user clicks a button:

```
document.getElementById("myBtn").addEventListener("click",
displayDate) ;
```

The `addEventListener()` method attaches an event handler to the specified element.

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element.

You can add many event handlers of the same type to one element, i.e two "click" events.

You can add event listeners to any DOM object not only HTML elements. i.e the window object.

The `addEventListener()` method makes it easier to control how the event reacts to bubbling.

When using the `addEventListener()` method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup.

You can easily remove an event listener by using the `removeEventListener()` method.

Syntax

```
element.addEventListener(event, function, useCapture);
```

The first parameter is the type of the event (like "`click`" or "`mousedown`" or any other [HTML DOM Event](#).)

The second parameter is the function we want to call when the event occurs.

The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

Add an Event Handler to an Element

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript addEventListener()</h2>
```

`<p>This example uses the addEventListener() method to attach a click event to a button.</p>`

`<button id="myBtn">Try it</button>`

```
<script>
document.getElementById("myBtn").addEventListener("click",
function() {
    alert("Hello World!");
});
</script>
```

`</body>`

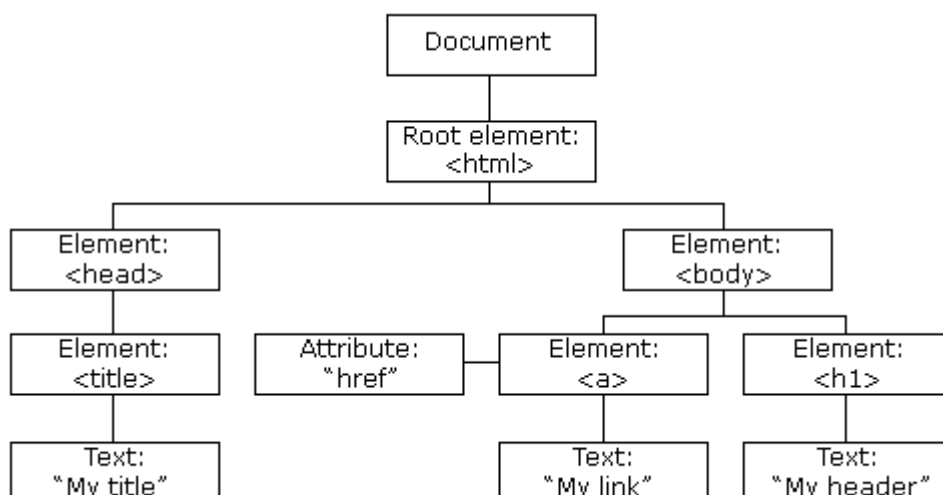
`</html>`

JavaScript HTML DOM Navigation

DOM Nodes

According to the W3C HTML DOM standard, everything in an HTML document is a node:

- The entire document is a document node
- Every HTML element is an element node
- The text inside HTML elements are text nodes
- Every HTML attribute is an attribute node (deprecated)
- All comments are comment nodes



Node Relationships

The nodes in the node tree have a hierarchical relationship to each other.

The terms parent, child, and sibling are used to describe the relationships.

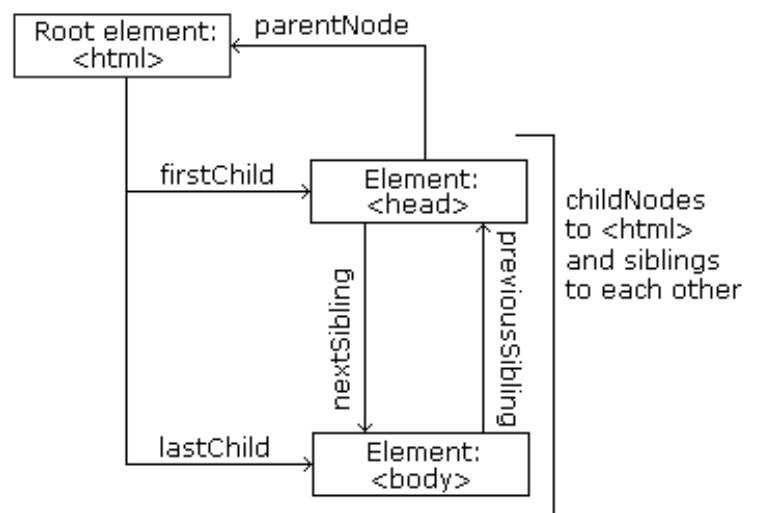
- In a node tree, the top node is called the root (or root node)
- Every node has exactly one parent, except the root (which has no parent)
- A node can have a number of children
- Siblings (brothers or sisters) are nodes with the same parent

```
<html>
```

```
  <head>
    <title>DOM
Tutorial</title>
  </head>

  <body>
    <h1>DOM Lesson one</h1>
    <p>Hello world!</p>
  </body>

</html>
```



From the HTML above you can read:

- `<html>` is the root node
- `<html>` has no parents
- `<html>` is the parent of `<head>` and `<body>`
- `<head>` is the first child of `<html>`
- `<body>` is the last child of `<html>`

and:

- `<head>` has one child: `<title>`

- `<title>` has one child (a text node): "DOM Tutorial"
- `<body>` has two children: `<h1>` and `<p>`
- `<h1>` has one child: "DOM Lesson one"
- `<p>` has one child: "Hello world!"
- `<h1>` and `<p>` are siblings

Navigating Between Nodes

You can use the following node properties to navigate between nodes with JavaScript:

- `parentNode`
- `childNodes[nodenum]`
- `firstChild`
- `lastChild`
- `nextSibling`
- `previousSibling`

Child Nodes and Node Values

Example

```
<html>
<body>

<h1 id="id01">My First Page</h1>
<p id="id02"></p>

<script>
document.getElementById("id02").innerHTML =
document.getElementById("id01").innerHTML;
</script>

</body>
</html>
```

Example

```
<html>
<body>

<h1 id="id01">My First Page</h1>
<p id="id02"></p>

<script>
document.getElementById("id02").innerHTML =
document.getElementById("id01").firstChild.nodeValue;
</script>

</body>
</html>
```

Example

```
<html>
<body>

<h1 id="id01">My First Page</h1>
<p id="id02">Hello!</p>

<script>
document.getElementById("id02").innerHTML =
document.getElementById("id01").childNodes[0].nodeValue;
</script>

</body>
</html>
```

InnerHTML

In this tutorial we use the innerHTML property to retrieve the content of an HTML element.

However, learning the other methods above is useful for understanding the tree structure and the navigation of the DOM.

DOM Root Nodes

There are two special properties that allow access to the full document:

- `document.body` - The body of the document
- `document.documentElement` - The full document

Example

```
<html>
<body>
<h2>JavaScript HTMLDOM</h2>
<p>Displaying document.body</p>

<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = document.body.innerHTML;
</script>
</body>
</html>
```

Example

```
<html>
<body>
<h2>JavaScript HTMLDOM</h2>
<p>Displaying document.documentElement</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
document.documentElement.innerHTML;
</script>
</body>
</html>
```

The nodeName Property

The **nodeName** property specifies the name of a node.

- **nodeName** is read-only
- **nodeName** of an element node is the same as the tag name
- **nodeName** of an attribute node is the attribute name
- **nodeName** of a text node is always #text
- **nodeName** of the document node is always #document

Example

```
<h1 id="id01">My First Page</h1>
<p id="id02"></p>
<script>
document.getElementById("id02").innerHTML =
document.getElementById("id01").nodeName;
</script>
```

Note: `nodeName` always contains the uppercase tag name of an HTML element.

The `nodeValue` Property

The `nodeValue` property specifies the value of a node.

- `nodeValue` for element nodes is `null`
- `nodeValue` for text nodes is the text itself
- `nodeValue` for attribute nodes is the attribute value

The `nodeType` Property

The `nodeType` property is read only. It returns the type of a node.

Example

```
<h1 id="id01">My First Page</h1>
<p id="id02"></p>
<script>
document.getElementById("id02").innerHTML =
document.getElementById("id01").nodeValue;
</script>
```

JavaScript HTML DOM Elements (Nodes)

Creating New HTML Elements (Nodes)

To add a new element to the HTML DOM, you must create the element (element node) first, and then append it to an existing element.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>
<p>Add a new HTML Element.</p>

<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);
const element = document.getElementById("div1");
element.appendChild(para);
</script>

</body>
</html>
```

Creating new HTML Elements - insertBefore()

The `appendChild()` method in the previous example, appended the new element as the last child of the parent.

If you don't want that you can use the `insertBefore()` method:

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const element = document.getElementById("div1");
const child = document.getElementById("p1");
element.insertBefore(para, child);
</script>
```

Removing Existing HTML Elements

To remove an HTML element, use the `remove()` method:

```
<div>
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const elmnt = document.getElementById("p1"); elmnt.remove();
</script>
```

Removing a Child Node

For browsers that does not support the `remove()` method, you have to find the parent node to remove an element:


```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>
```

```
<script>
const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.removeChild(child);
</script>
```

Replacing HTML Elements

To replace an element to the HTML DOM, use the `replaceChild()` method:

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>
```

```
<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.replaceChild(para, child);
</script>
```

JavaScript HTML DOM Collections

The HTMLCollection Object

The `getElementsByName()` method returns an `HTMLCollection` object.

An `HTMLCollection` object is an array-like list (collection) of HTML elements.

The following code selects all `<p>` elements in a document:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<p>Hello World!</p>

<p>Hello Norway!</p>

<p id="demo"></p>

<script>
const myCollection = document.getElementsByTagName("p");

document.getElementById("demo").innerHTML = "The innerHTML of
the second paragraph is: " + myCollection[1].innerHTML;

</script>

</body>
</html>
```

JavaScript HTML DOM Node Lists

The HTML DOM NodeList Object

A **NodeList** object is a list (collection) of nodes extracted from a document.

A **NodeList** object is almost the same as an **HTMLCollection** object.

Some (older) browsers return a **NodeList** object instead of an **HTMLCollection** for methods like **getElementsByClassName()**.

All browsers return a **NodeList** object for the property **childNodes**.

Most browsers return a **NodeList** object for the method **querySelectorAll()**.

The following code selects all **<p>** nodes in a document:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<p>Hello World!</p>

<p>Hello Norway!</p>

<p id="demo"></p>

<script>
const myNodelist = document.querySelectorAll("p");

document.getElementById("demo").innerHTML = "This document
contains " + myNodelist.length + " paragraphs.";

</script>

</body>
</html>
```

JavaScript Errors

Throw, and Try...Catch...Finally

The **try** statement defines a code block to run (to try).

The **catch** statement defines a code block to handle any error.

The **finally** statement defines a code block to run regardless of the result.

The **throw** statement defines a custom error.

JavaScript try and catch

The **try** statement allows you to define a block of code to be tested for errors while it is being executed.

The **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.

The JavaScript statements **try** and **catch** come in pairs:

```
try {  
    Block of code to try  
}  
catch(err) {  
    Block of code to handle errors  
}
```

JavaScript Throws Errors

When an error occurs, JavaScript will normally stop and generate an error message.

The technical term for this is: JavaScript will throw an exception (throw an error).

The throw Statement

The **throw** statement allows you to create a custom error.

Technically you can throw an exception (throw an error).

The exception can be a JavaScript **String**, a **Number**, a **Boolean** or an **Object**:

Input Validation Example

```
<!DOCTYPE html>
<html>
<body>

<p>Please input a number between 5 and 10:</p>

<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test
Input</button>
<p id="p01"></p>
```

```
<script>
function myFunction() {
  const message = document.getElementById("p01");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;
  try {
    if(x.trim() == "") throw "empty";
    if(isNaN(x)) throw "not a number";
    x = Number(x);
    if(x < 5) throw "too low";
    if(x > 10) throw "too high";
  }
  catch(err) {
    message.innerHTML = "Input is " + err;
  }
}
</script>

</body>
</html>
```

The finally Statement

The **finally** statement lets you execute code, after try and catch, regardless of the result:

Syntax

```
try {
  Block of code to try
}
catch(err) {
  Block of code to handle errors
}
finally {
```

```
    Block of code to be executed regardless of the try / catch
    result
}
```

Example

```
function myFunction() {
    const message = document.getElementById("p01");
    message.innerHTML = "";
    let x = document.getElementById("demo").value;
    try {
        if(x.trim() == "") throw "is empty";
        if(isNaN(x)) throw "is not a number";
        x = Number(x);
        if(x > 10) throw "is too high";
        if(x < 5) throw "is too low";
    }
    catch(err) {
        message.innerHTML = "Error: " + err + ".";
    }
    finally {
        document.getElementById("demo").value = "";
    }
}
```

JavaScript Events

HTML events are "things" that happen to HTML elements.

When JavaScript is used in HTML pages, JavaScript can "react" on these events.

HTML Events

An HTML event can be something the browser does, or something a user does.

Here are some examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

Often, when events happen, you may want to do something.

JavaScript lets you execute code when events are detected.

HTML allows event handler attributes, with JavaScript code, to be added to HTML elements.

With single quotes:

```
<element event='some JavaScript'>
```

With double quotes:

```
<element event="some JavaScript">
```

In the following example, an `onclick` attribute (with code), is added to a `<button>` element:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<h2>The onclick Attribute</h2>

<button
onclick="document.getElementById('demo').innerHTML=Date()">
The time is?</button>

<p id="demo"></p>

</body>
</html>
```